

spaCraft: Analysis-aware power calculation and sample size planning for multi-sample spatial transcriptomics

Overview

spaCraft provides a **pilot-based framework for statistical power analysis and sample-size determination** in comparative, multi-sample spatial transcriptomics studies. Given one or more annotated pilot samples, it learns a cohort-level generative model and then estimates, by Monte Carlo, how many samples per group are needed to detect an effect of a given size — for two distinct scientific questions:

- **SaLFC** — a *spatially adjusted* differential-expression test for a Target-vs-Reference log-fold-change between two groups of samples; and
- **LOR** — a baseline-anchored *compositional* test for a shift in the abundance of a target domain between groups.

The engine is a **generate–recover–test** pipeline. A three-layer generative model is fitted from pilot data — a baseline-anchored Binomial–logit model for domain *composition*, a Fisher–Gaussian kernel mixture model (FGKMM) for spatial *geometry*, and a Gaussian-process (NNGP) model for *expression*. Synthetic cohorts are then simulated under a user-specified effect size, spatial domains are recovered with a pilot-guided extension of BANKSY (**pBANSKY**), and the chosen endpoint is applied. Repeating this over a grid of sample sizes K and effect sizes yields empirical power, which is smoothed with a monotone shape-constrained additive model to recommend a per-group K .

In this vignette we walk through the **entire workflow on human dorsolateral prefrontal cortex (DLPFC) Visium data** (Maynard et al., 2021, obtained via `spatialLIBD`), from a pilot list all the way to SaLFC and LOR power curves.

Running this vignette. Rather than loading a separate precomputed object at each stage, this vignette **starts from the bundled, downsampled pilot list `visium_human_brain_pilot_data_list` and builds every subsequent object in sequence** — `createspaCraftObject()` produces the working object, feature selection adds the gene sets, the three estimators fill the parameter slots, the simulator generates a cohort, and `pBANSKY()` recovers it. Each step simply consumes the object returned by the previous one, so there is **no need to call `data(mini_obj_features)`, `data(mini_obj_fitted)`, and the like**; those bundled objects are just snapshots of this same chain at different points. The whole pipeline through a single `simulate` → `recover` → `test` pass runs live here, single-threaded (`n_cores = 1`). Two caveats follow from using a tiny dataset: (i) because it is heavily downsampled (a few hundred genes and spots), the feature-selection cutoffs are relaxed accordingly and the selected genes and reported numbers are **illustrative, not the full-data results from the paper**; and (ii) the Monte-Carlo **hyperparameter-tuning and power-grid sweeps** are shown with small settings but left **unevaluated** (`eval = FALSE`) so the vignette builds quickly — flip them to `eval = TRUE` (and raise `n_sim`) to reproduce them.

```
library(spaCraft)
```

The pilot data

A pilot dataset is a plain `list` of samples. Each element is itself a list with a `counts` matrix (genes × spots), a `coords` data frame (with columns `x`, `y`, and the annotated `domain`), a `group` label (`control = 0`, `case =`

1), and a `sample_id`. This is exactly the format you would assemble from a `SpatialExperiment`/`Seurat` object or from `spatialLIBD`. For your own analysis you would load such a list and pass it straight to `createspaCraftObject()`:

```
# For a real analysis: a list of samples, each with counts, coords (x, y,
# domain), group, and sample_id.
load("path/to/visium_human_brain_pilot_data_list.RData") # provides `pilot_data_list`
spaCraft_obj <- createspaCraftObject(pilot_data_list)
```

Here we use the bundled, downsampled DLPFC pilot list and build the object from it. This is the first and only place we touch a `mini_*` dataset — everything downstream grows out of `spaCraft_obj`.

```
data(visium_human_brain_pilot_data_list)

length(visium_human_brain_pilot_data_list) # number of pilot samples
str(visium_human_brain_pilot_data_list[[1]], max.level = 1)

spaCraft_obj <- createspaCraftObject(visium_human_brain_pilot_data_list)
```

`createspaCraftObject()` validates the list and, if a group has only a single replicate ($K = 1$), can optionally generate pseudo-replicates so that the downstream multi-sample estimators remain well defined (see `generate_pseudo_reps`). The result is the S4 object that every later step reads from and writes to.

Feature selection

`spaCraft` works with two complementary gene sets, identified in two passes. `featureSelection()` performs a multi-context meta-analysis to find robust **domain-informative markers** (`top_genes`), while `featureSelectionStable()` identifies **spatially and group-wise stable genes** (`stable_genes`) that serve as an empirical null and as the pool from which spike-in genes are later drawn. Because the bundled data is downsampled, we deliberately **relax the cutoffs** so that genes survive on a few hundred features; on full data you would use stricter thresholds (e.g. `logfc_cutoff = 0.5`, `mean_in_cutoff = 1`).

```
spaCraft_obj <- featureSelection(
  object      = spaCraft_obj,
  logfc_cutoff = 0.01,
  mean_in_cutoff = 0.01,
  max_num_gene = 10,
  combine_method = "union",
  n_cores      = 1
)

spaCraft_obj <- featureSelectionStable(
  object          = spaCraft_obj,
  control_group_value = "0",
  case_group_value  = "1",
  check_group_diff  = TRUE,
  max_group_logfc_cut = 0.5,
  mean_global_cut    = 0.1,
  pct_in_cut         = 0.1,
  min_domain_prop    = 0.7,
  max_abs_logfc_cut   = 0.5,
  cv_cut            = 0.8,
```

```

    max_num_gene      = 50,
    use_low           = TRUE,
    n_cores           = 1,
    verbose           = TRUE
  )

  head(spaCraft_obj@params_expression$top_genes)
  head(spaCraft_obj@params_expression$stable_genes)

```

Why two passes? The markers (`top_genes`) carry the spatial signal that lets pBANSKY recover domains, whereas the stable genes (`stable_genes`) are deliberately *uninformative* — flat across space and across groups — so that injecting a controlled effect into a few of them produces a clean, known alternative against an honest null.

Defining the simulation gene sets

`makeCustomGeneSets()` turns the two selected sets into the three gene sets the simulator and the endpoints consume, for a chosen Target/Reference domain pair (here white matter vs. layer 6):

- `G_svg` — spatially variable genes (the markers plus stable genes) used to *label* spots;
- `G_null` — the stable / empirical-null genes; and
- `G_spike` — the top differentially expressed genes between Target and Reference, used for *signal injection*.

```

library(spaCraft)
load("/Users/shin.991/Library/CloudStorage/Dropbox/postdoc/research/main_project/spaCraft/spaCraft_pack
load("/Users/shin.991/Library/CloudStorage/Dropbox/postdoc/research/main_project/spaCraft/spaCraft_pack
spaCraft_obj <- spaDesign2_obj

G_svg_base <- spaCraft_obj@params_expression$top_genes
G_stable   <- spaCraft_obj@params_expression$stable_genes

sets <- makeCustomGeneSets(
  object      = spaCraft_obj,
  G_svg_base  = G_svg_base,
  G_stable    = G_stable,
  target_domain = "WM",
  reference_domain = "Layer6",
  sample_index = 1L,
  n_de        = 5
)

lengths(sets[c("G_svg", "G_null", "G_spike")])
#>   G_svg  G_null  G_spike
#>    31    20     5
#head(sets$TR_stats, n=3)

```

Fitting the three-layer generative model

With gene sets in hand we fit the cohort-level model. The three layers are independent calls and each writes into its own slot of `spaCraft_obj`.

Expression (NNGP). `estimateExpressionParams_v2()` estimates per-gene spatial covariance and biological variance across samples using nearest-neighbour Gaussian-process approximations (Saha & Datta, 2018), accelerated with Rcpp.

```
spaCraft_obj <- estimateExpressionParams_v2(
  object          = spaCraft_obj,
  target_domain   = "WM",
  reference_domain = "Layer6",
  genes_to_use    = sets$G_svg,
  n_neighbors     = 15,
  n_cores         = 1,
  verbose        = TRUE
)
```

Geometry (FGKMM). `estimateGeometryParams_v2()` pools each domain's spot cloud across samples and fits a Fisher–Gaussian kernel mixture (Mukhopadhyay, Li & Dunson, 2020), selecting the number of kernels by BIC over `M_candidates`.

```
spaCraft_obj <- estimateGeometryParams_v2(
  object          = spaCraft_obj,
  M_candidates    = 3:6,
  iter_max       = 1000,
  tol            = 1e-2,
  n_cores        = 1,
  verbose        = TRUE
)
```

Composition (Binomial–logit). `estimateCompositionParams_v2()` fits the baseline-anchored model for the Target-vs-Reference abundance; the pilot baseline β_1 becomes the null center used by the LOR TREAT test.

```
spaCraft_obj <- estimateCompositionParams_v2(
  object          = spaCraft_obj,
  target_domain   = "WM",
  reference_domain = "Layer6",
  verbose        = TRUE
)
```

All three parameter slots are now populated. (This is exactly the state captured by the bundled `mini_obj_fitted` object — but here we built it ourselves from the pilot list.)

```
names(spaCraft_obj@params_composition) # baseline-anchored Binomial-logit
names(spaCraft_obj@params_expression)  # gene sets + NNGP parameters
names(spaCraft_obj@params_geometry)    # FGKMM geometry
```

We can visualize the pilot domains directly from the object. The `S4 plot()` method draws spatial domains with a consistent palette across samples:

```
plot(spaCraft_obj, type = "pilot", n_per_group = 2)
```

(Optional) Stability-driven hyperparameter tuning

The recover step has two spatial knobs: the conditional-texture strength of the simulator (`lambda_cond`) and the BANKSY spatial-mixing weight of the clusterer (`lambda_p`). `tuneSpaDesignLambdas()` selects them by stability under a single H_1 scenario, returning endpoint-specific optima for SaLFC and LOR. This is a Monte-Carlo search, so it is shown here with small settings but **not run during the build** (set `eval = TRUE` to run it).

```
tuning_H1_scenario <- list(
  DE_lfc = 0.3, target_prop_case = 0.7, kappa_geo = 1.0,
  kappa_m = 1.0, kappa_bio = 1.0, delta_rho = 1.0
)

tuning_obj <- tuneSpaDesignLambdas(
  object      = spaCraft_obj,
  target_domain = "WM", reference_domain = "Layer6",
  G_DE        = sets$G_spike,
  scenario_H1  = tuning_H1_scenario,
  lambda_p_grid = seq(0.3, 0.7, by = 0.1), # small grid for speed
  lambda_c_grid = seq(0.3, 0.7, by = 0.1),
  B = 20, K_syn = 10, n_cores = 1, seed = 123
)

summary(tuning_obj)
plot(tuning_obj)
```

The values selected here (`tuning_obj$optimal_params$SaLFC$lambda_p`, etc.) can be passed to the power functions below; if you skip tuning, just use fixed values, as we do next.

Simulating a synthetic cohort

One synthetic cohort makes the pipeline concrete. The scenario list controls the effect being injected; here we turn expression DE *off* (`DE_lfc = 0`) and raise the case-group target proportion to 0.7, i.e. a purely compositional effect.

```
class(spaCraft_obj) <- "spaCraft"

scenario_params <- list(
  DE_lfc      = 0.3, # expression log-fold-change for DE genes
  target_prop_case = NULL, # case-group target-domain proportion
  kappa_geo = 1.0, kappa_m = 1.0, kappa_bio = 1.1,
  delta_rho = 1.0, lambda_cond = tuning_obj$optimal_params$SaLFC$lambda_cond
)

sim_results <- simulatespaCraft(
  object      = spaCraft_obj,
  n_sample_per_group = 5,
  scenario_settings = scenario_params,
  target_domain    = "WM",
  genes_to_simulate = sets$G_svg,
  de_genes         = sets$G_spike,
  spatial_mode      = "original",
  graph_k           = 30L,
```

```
n_cores      = 1
)
```

Recovering spatial domains and visualizing

`pBANSKY()` recovers domain labels on the synthetic samples using a pilot-centroid-guided BANKSY embedding; the recovered labels are written to `coords$hat_d`. We then re-shape the synthetic blobs onto the pilot geometry with `rearrangeSyntheticToPilot()` and store the result back in the object.

```
syn_d_hat <- pBANSKY(
  sim_data = sim_results,
  pilot_data = spaCraft_obj@pilot_data,
  lambda = tuning_obj$optimal_params$SaLFC$lambda_p, k_neighbors = 30,
  scale_pilot_coords = FALSE, scale_sim_coords = FALSE,
  hard_guidance = FALSE, n_pcs = 20, do_hungarian = TRUE,
  use_G = TRUE, balance_mode = "energy_proportional"
)
#> /

# pack into the {counts, coords, meta} shape the downstream steps expect
synthetic_data <- setNames(
  lapply(syn_d_hat, function(item) {
    list(counts = item$counts, coords = item$coords,
         meta = data.frame(sample_id = item$sample_id, group = item$group))
  }),
  vapply(syn_d_hat, function(item) item$sample_id, character(1))
)

mapped_synthetic_data <- rearrangeSyntheticToPilot(
  synthetic_data = synthetic_data,
  pilot_data = spaCraft_obj@pilot_data,
  boundary_shift = TRUE, # render a composition shift as a moving boundary
  target_domain = "WM",
  reference_domain = "Layer6"
)

syntheticData(spaCraft_obj) <- mapped_synthetic_data
```

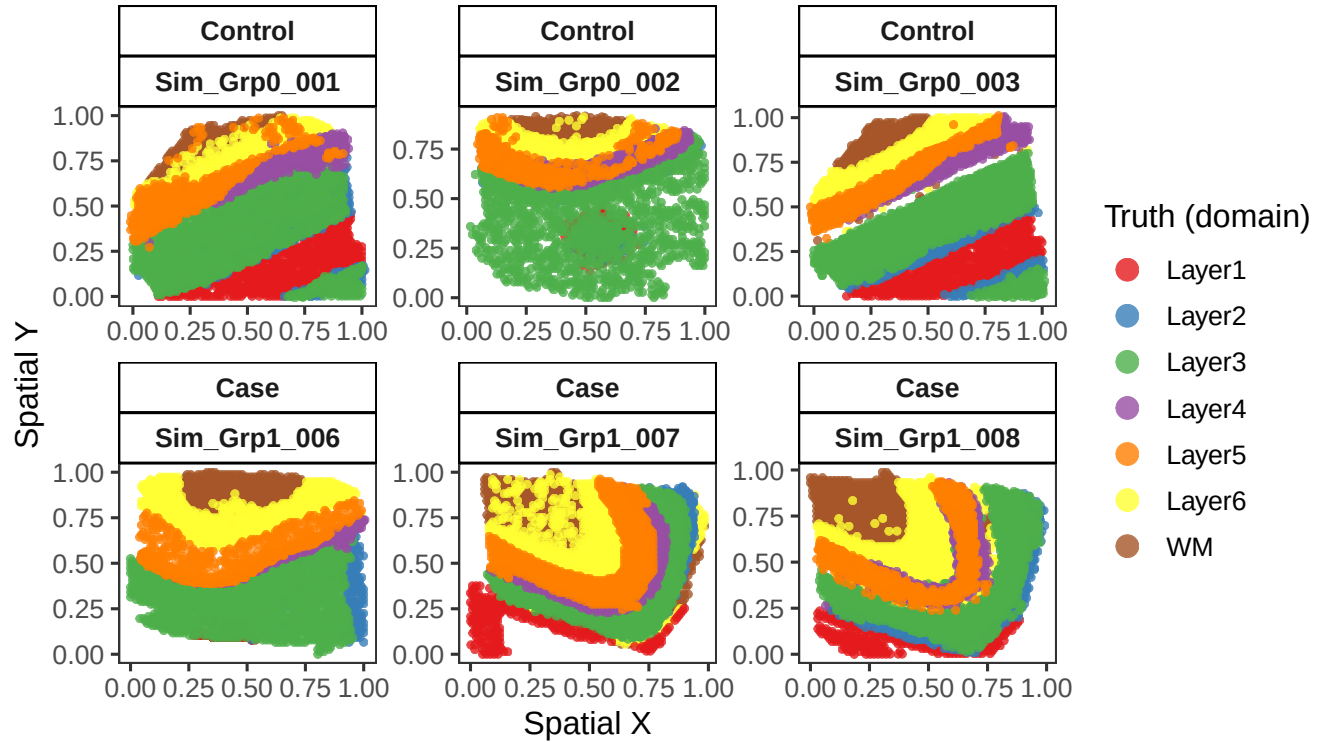
lambda and boundary_shift. `lambda` is BANKSY's spatial-mixing weight (0 = expression only, 1 = neighbourhood only); this is the `lambda_p` you would tune. `boundary_shift = TRUE` is appropriate for the LOR (composition) endpoint: it renders a larger case-group target proportion as the target band expanding into the reference, leaving per-domain counts (and hence the LOR statistic) untouched while only the spatial footprint changes.

Now that synthetic data are stored in the object, the same `plot()` method shows the raw simulated domains and the pilot-mapped versions:

```
plot(spaCraft_obj, type = "synthetic_mapped",
     color_by = "domain", n_per_group = 3, cex = 1.2, alpha = 0.8)
```

Synthetic Data: (After Rearrangement)

Top Row: Control | Bottom Row: Case



A single-design-point test

Before sweeping a grid, it is instructive to apply each endpoint **once** to the single synthetic cohort we just recovered. This exposes the raw test output and is exactly the computation that `evaluatePowerSaLFC()` and `evaluatePowerLOR()` repeat internally at every (K, effect) cell. We pass `synthetic_data` — the recovered cohort carrying cluster labels in `coords$hat_d` — and let both tests derive their TREAT thresholds directly from the pilot (`lfc_threshold = "pilot"`, `tau = "pilot"`).

SaLFC. Run here on the null genes, so the p-values serve as a size check — recall that this cohort was simulated with `DE_lfc = 0` (a purely compositional effect), so no expression signal was injected:

```
res_SaLFC <- SaLFC(
  synthetic_data, spaCraft_obj,
  genes          = sets$G_null,
  target_domain  = "WM", reference_domain = "Layer6",
  domain_col     = "hat_d",
  lfc_threshold  = "pilot"
)

sets$G_null      # empirical-null genes
sets$G_spike     # injected-DE genes (relevant when DE_lfc > 0)
round(res_SaLFC$p_value, 5)
```

LOR. A single composition test for the WM-vs-Layer6 abundance; this is the endpoint expected to register the injected `target_prop_case = 0.7` shift:

```
res_LOR <- LOR(
  synthetic_data, spaCraft_obj,
  target_domain = "WM", reference_domain = "Layer6",
  null_type = "treat", tau = "pilot"
)

res_LOR$p_value
```

Each rejection (`p_value <= alpha`) is a single Bernoulli outcome; the power functions below simply average many such outcomes across the (K , effect) grid.

Power analysis

We are now ready for the headline output: power as a function of the per-group sample size K . Each grid point runs the full simulate → recover → test pipeline `n_sim` times; the effect = 0 row is the empirical size/FDR. These sweeps are the expensive part of the package, so they are shown here with small grids but **left unevaluated**; set `eval = TRUE` and raise `n_sim` (e.g. 100–500) for a real study. We pass fixed spatial weights below; in a full analysis you would substitute the tuned values, e.g. `clustering_args = list(lambda = tuning_obj$optimal_params$SaLFC$lambda_p)`.

SaLFC — spatially adjusted differential expression

The swept effect is the expression log-fold-change `lfc_grid`. Pilot-based TREAT thresholds are estimated once per effect size and shared across replicates.

```
# Note: This evaluation is computationally intensive.
power_results <- evaluatePowerSaLFC(
  object = spaCraft_obj,
  K_grid = seq(3, 9, by=1),
  lfc_grid = c(0.30, 0.40, 0.50),
  n_sim = 10,
  seed = 2026,
  G_svg = sets$G_svg,
  G_null = sets$G_null,
  G_spike = sets$G_spike,
  scenario_base = list(lambda_cond = tuning_obj$optimal_params$SaLFC$lambda_cond),
  clustering_args = list(lambda = tuning_obj$optimal_params$SaLFC$lambda_p)
)
```

```
library(dplyr); library(ggplot2); library(ggrepel); library(scam)
library(spaCraft)
# Load pre-computed power results from the package to expedite vignette rendering
salfc_csv_url <- "/Users/shin.991/Library/CloudStorage/Dropbox/postdoc/research/main_project/spaCraft/salfc_csv"
power_results <- read.csv(salfc_csv_url)

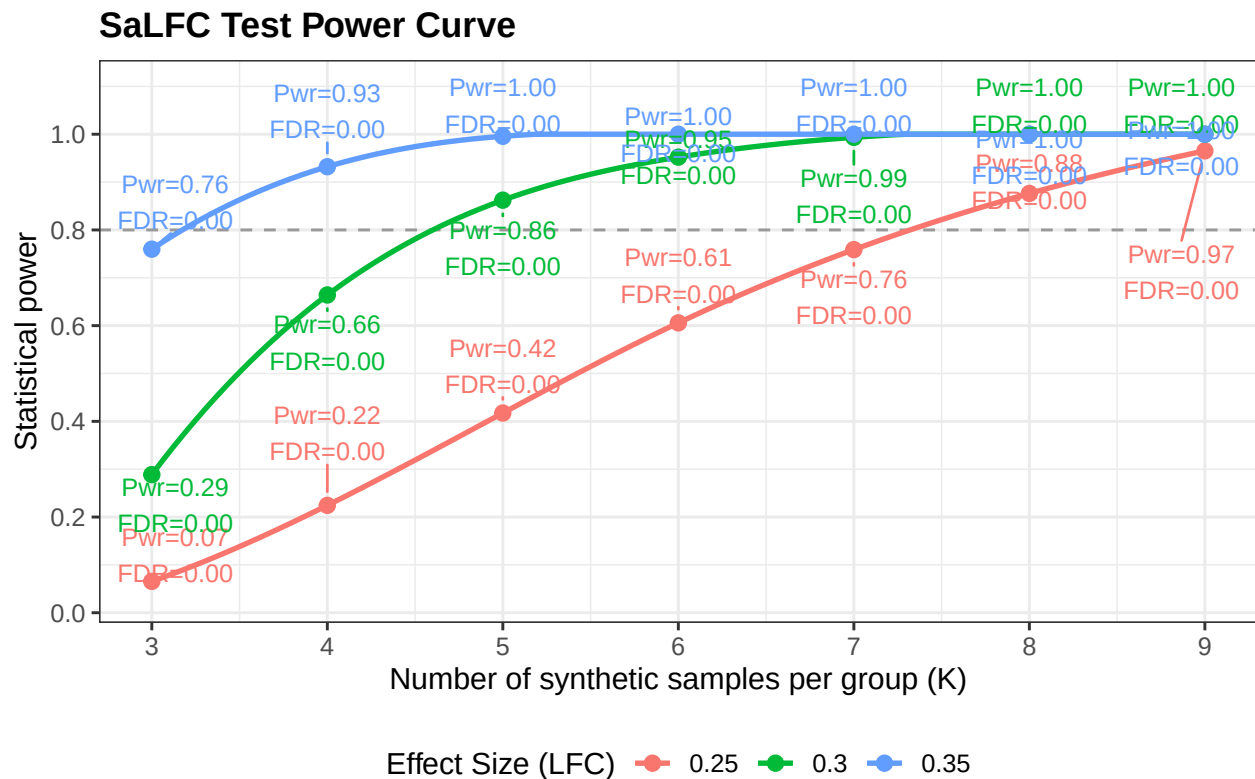
print(head(power_results))
#>   K_syn DE_lfc adjust_method test_method eval_domain_col rearrange strict_ok
#> 1     3   0.25   bonferroni           t           hat_d      TRUE      TRUE
#> 2     3   0.25   bonferroni           t           hat_d      TRUE      TRUE
#> 3     3   0.25   bonferroni           t           hat_d      TRUE      TRUE
#> 4     3   0.25   bonferroni           t           hat_d      TRUE      TRUE
```



```

#> 5      3 0.25 bonferroni      t      hat_d TRUE TRUE
#> 6      3 0.25 bonferroni      t      hat_d TRUE TRUE
#> power_strict power_raw fdp n_rej n_TP n_FP n_test n_DE spatial_mode rep
#> 1      0.0      0.0 0 0 0 0 10 5 original 1
#> 2      0.0      0.0 0 0 0 0 10 5 original 2
#> 3      0.0      0.0 0 0 0 0 10 5 original 3
#> 4      0.0      0.0 0 0 0 0 10 5 original 4
#> 5      0.2      0.2 0 1 1 0 10 5 original 5
#> 6      0.0      0.0 0 0 0 0 10 5 original 6
#> seed_used
#> 1      10201
#> 2      10202
#> 3      10203
#> 4      10204
#> 5      10205
#> 6      10206
plotPowerCurveSaLFC(power_results, alpha = 0.05)

```



LOR — compositional shift

Here the swept effect is the case-group target proportion (`effect_type = "target_prop_case"`); expression DE is switched off internally so the signal is purely compositional. LOR returns a single test per replicate, so power is simply the rejection rate.

```

# Note: This evaluation is computationally intensive.
lor_results <- evaluatePowerLOR(
  object      = spaCraft_obj,

```

```

K_grid      = seq(3, 9, by=1),
delta_grid  = c(0.60, 0.70, 0.80),
effect_type = "target_prop_case",
n_sim       = 10,
seed        = 2026,
genes       = sets$G_svg,
scenario_base = list(lambda_cond = tuning_obj$optimal_params$LOR$lambda_cond),
clustering_args = list(lambda = tuning_obj$optimal_params$LOR$lambda_p)
)

```

```

library(dplyr); library(ggplot2); library(ggrepel); library(scam)
library(spaCraft)
lor_csv_url <- "/Users/shin.991/Library/CloudStorage/Dropbox/postdoc/research/main_project/spaCraft/spaCraft.csv"
lor_results <- read.csv(lor_csv_url)

```

```
print(head(lor_results))
```

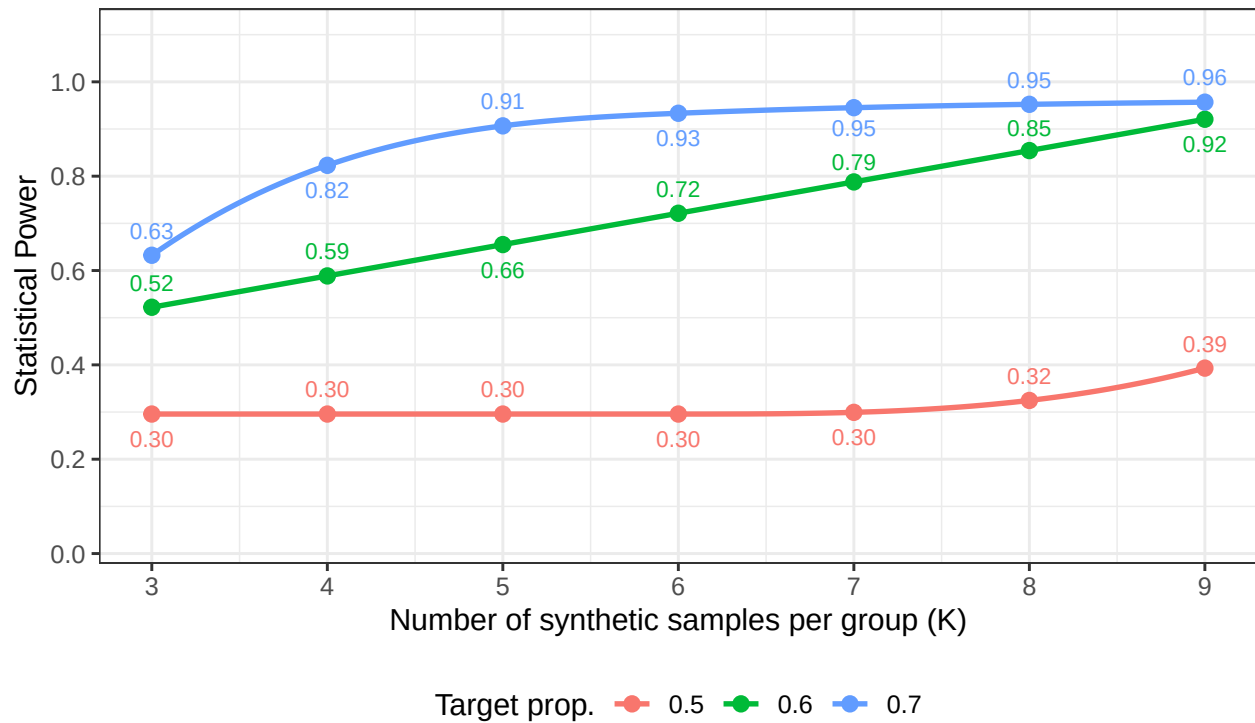
```

#>   K_syn effect      effect_type delta_pp target_prop_case null_type eval_on
#> 1     3   0.5 target_prop_case      NA             0.5      treat   hat_d
#> 2     3   0.5 target_prop_case      NA             0.5      treat   hat_d
#> 3     3   0.5 target_prop_case      NA             0.5      treat   hat_d
#> 4     3   0.5 target_prop_case      NA             0.5      treat   hat_d
#> 5     3   0.5 target_prop_case      NA             0.5      treat   hat_d
#> 6     3   0.5 target_prop_case      NA             0.5      treat   hat_d
#>   test_method reject   p_value delta_hat   delta0 delta_tilde   tau KO K1
#> 1           t FALSE 0.3650292 0.4307171 0.1927264 0.2379907 0.1350878 3 3
#> 2           t FALSE 0.3510681 0.4520862 0.1927264 0.2593598 0.1350878 3 3
#> 3           t FALSE 0.4646511 0.3541249 0.1927264 0.1613985 0.1350878 3 3
#> 4           t  TRUE 0.0102806 0.8093831 0.1927264 0.6166567 0.1350878 3 3
#> 5           t FALSE 0.2302085 0.5244662 0.1927264 0.3317398 0.1350878 3 3
#> 6           t FALSE 0.2062331 0.5407977 0.1927264 0.3480713 0.1350878 3 3
#>   rep seed_used
#> 1   1    10201
#> 2   2    10202
#> 3   3    10203
#> 4   4    10204
#> 5   5    10205
#> 6   6    10206

```

```
plotPowerCurveLOR(lor_results, alpha = 0.05)
```

LOR Composition Power / Size Curve ($\alpha = 0.05$)



Reading the curves. Each line is one effect size; the dashed reference line is the nominal level α . The `summary()` reports, for each (non-null) effect, the smallest K reaching the target power — the per-group sample size you would recommend — together with a size/FDR diagnostic at the null.

Session info

```
sessionInfo()
```